

Spark Partition 학습

Shuffling과 Partition에 대해 더 배워보자

Grepp, Inc

한기용

keeyonghan@hotmail.com

Contents

1. Dynamically optimizing skew joins
2. Salting을 통한 Data Skew 처리
3. All about Partitions



Dynamically optimizing skew joins

◆ Dynamically optimizing skew joins는 무엇인가?

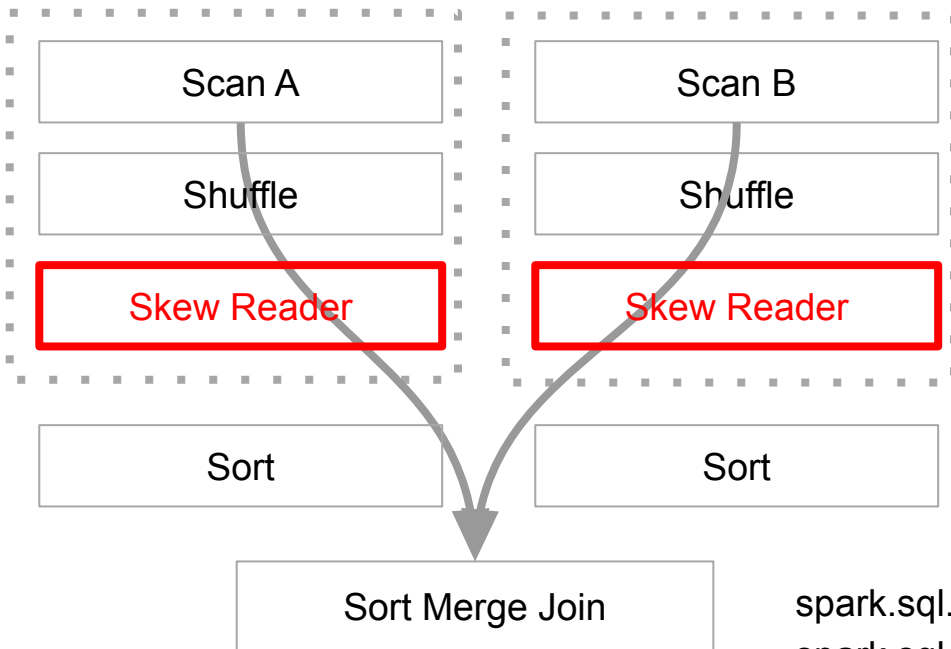
❖ 먼저 왜 필요한가?

- **Skew** 파티션으로 인한 성능 문제를 해결하기 위함
 - 한 두개의 오래 걸리는 태스크들로 인한 전체 **Job/Stage** 종료 지연
 - 이 때 **disk spill**이 발생한다면 더 느려지게 됨

❖ AQE의 해법

- 먼저 **skew** 파티션의 존재 여부 파악
- 다음으로 **skew** 파티션을 작게 나누고
- 다음으로 상대 조인 파티션을 중복하여 만들고 조인 수행

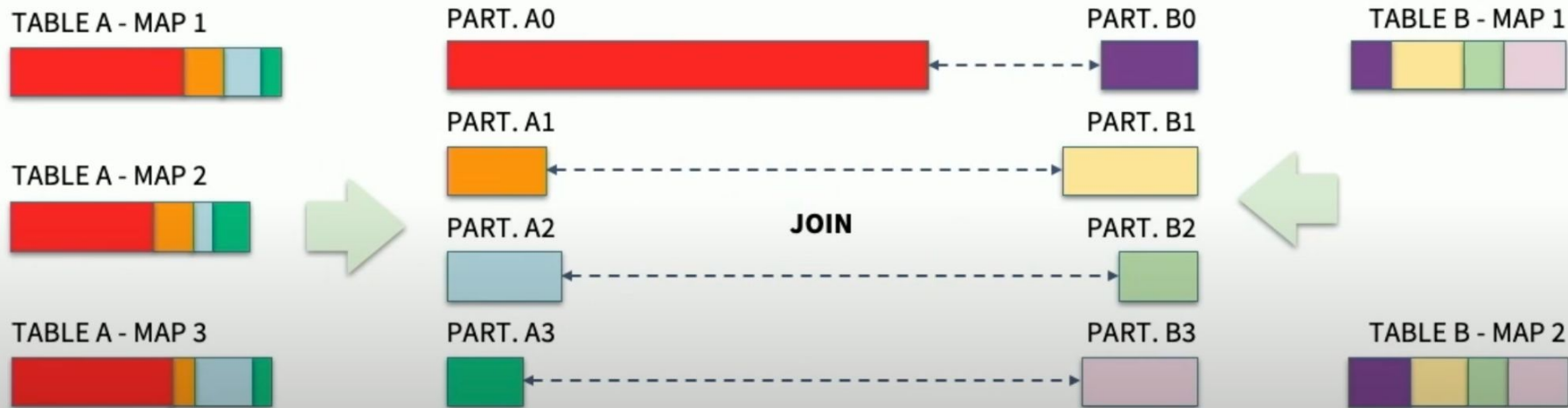
◆ Dynamically optimizing skew joins 동작방식



1. Leaf stage 실행
2. 한쪽에 skew partition이 보이는 경우 skew reader를 통해 작은 다수의 파티션으로 재구성
3. 조인 대상이 되는 반대편 파티션은 앞서 다수의 부분 파티션쪽으로 중복해서 파티션을 생성

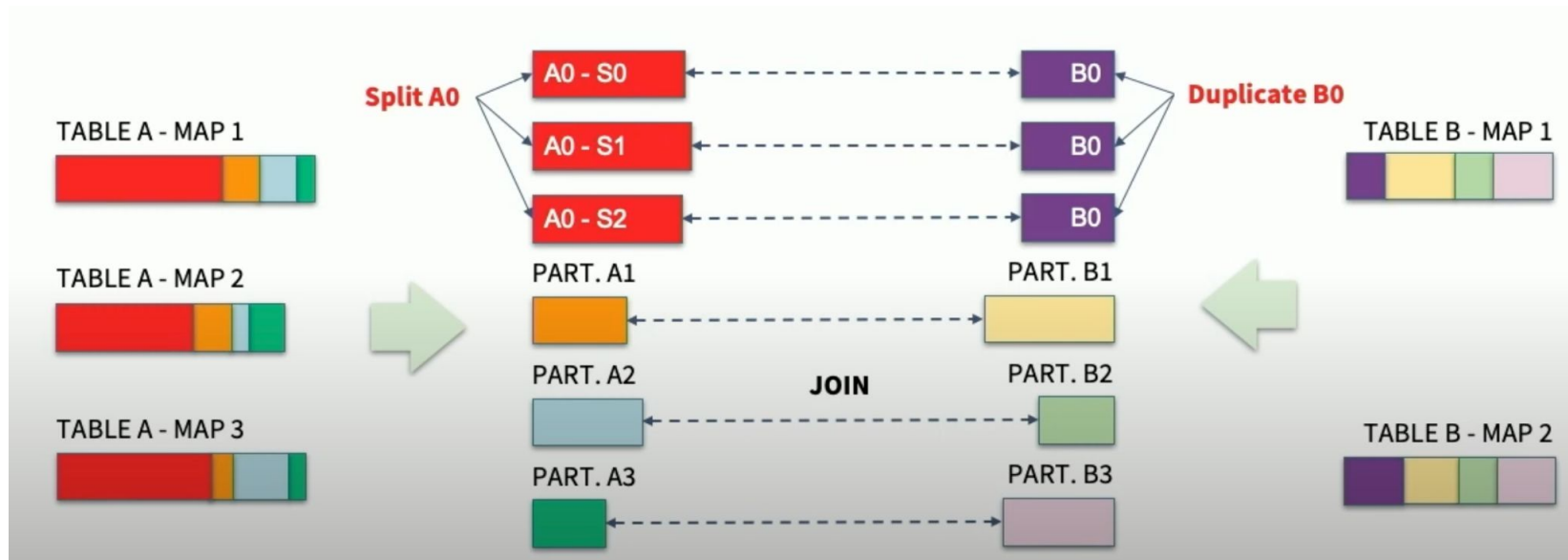
`spark.sql.adaptive.skewJoin.skewedPartitionFactor`
`spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes`

◆ Dynamically optimizing skew joins 그림 (1)



Source: Databricks

◆ Dynamically optimizing skew joins 그림 (2)



Source: Databricks

◆ Dynamically optimizing skew joins 환경 변수들 (3.3.1)

환경 변수 이름	기본값	설명
spark.sql.adaptive.skewJoin.enabled	True	이 값과 spark.sql.adaptive.enabled가 true인 경우, Sort Merge Join 수행시 skew된 파티션들을 먼저 크기를 줄이고 난 다음에 조인 수행
spark.sql.adaptive.skewJoin.skewedPartitionFactor	5	파티션의 크기가 중간 파티션 크기보다 5배 이상크고 spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes의 값보다 크면 skew 파티션으로 간주
spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes	256MB	파티션의 크기가 이 값보다 크고 중간 파티션 크기보다 5배 이상 크면 skew 파티션으로 간주. 이 값은 spark.sql.adaptive.advisoryPartitionSizeInBytes보다 커야함.

◆ 데모에서 사용했던 SQL 정리 (1)

```
CREATE TABLE items
```

```
USING parquet
```

```
AS
```

```
SELECT id,
```

```
CAST(rand() * 1000 AS INT) AS price
```

```
FROM RANGE(300000000);
```

```
CREATE TABLE sales
```

```
USING parquet
```

```
AS
```

```
SELECT CASE WHEN rand() < 0.8 THEN 100 ELSE CAST(rand() * 300000000 AS INT) END AS item_id,
```

```
CAST(rand() * 100 AS INT) AS quantity,
```

```
DATE_ADD(current_date(), - CAST(rand() * 360 AS INT)) AS date
```

```
FROM RANGE(10000000000);
```

◆ 데모에서 사용했던 SQL - Coalescing

```
SELECT date, sum(quantity) AS q  
FROM sales  
GROUP BY 1  
ORDER BY 2 DESC;
```

```
pyspark --driver-memory 2g --executor-memory 2g
```

◆ 데모에서 사용했던 SQL - JOIN Strategies

```
SELECT date, sum(quantity * price) AS total_sales  
FROM sales s  
JOIN items i ON s.item_id = i.id  
WHERE price < 10  
GROUP BY 1  
ORDER BY 2 DESC;
```

```
pyspark --driver-memory 2g --executor-memory 2g
```

◆ 데모에서 사용했던 SQL - SKEW JOIN optimization

```
SELECT date, sum(quantity * price) AS total_sales  
FROM sales s  
JOIN items i ON s.item_id = i.id  
GROUP BY 1  
ORDER BY 2 DESC;
```

```
pyspark --driver-memory 4g --executor-memory 8g
```

Salting을 통한 Data Skew 처리

AQE가 나오기 전 data skew 처리 방식 중의 하나는
Salting이었는데 이에 대해 알아보자

◆ 파티션 관련 환경 설정 변수들 (3.3.1 기준)

환경 변수 이름	기본값	설명
<code>spark.sql.adaptive.enabled</code>	True	Spark 3.2부터 기본으로 적용되기 시작 (처음 도입은 1.6)
<code>spark.sql.shuffle.partitions</code>	200	Shuffle 후 만들어질 파티션의 기본 수. 뒤에 좀더 이야기
<code>spark.sql.files.maxPartitionBytes</code>	128MB	파일 기반 데이터 소스(Parquet, JSON, ORC 등등)에서 데이터프레임을 만들 때 한 파티션의 최대 크기
<code>spark.sql.files.openCostInBytes</code>	4MB	이 역시 파일 기반 데이터소스에서만 의미가 있으며 입력 데이터를 읽기 위해 몇 개의 파티션을 사용할지 결정하는데 사용됨. 대부분의 경우 별 의미 없는 변수이지만 기본적으로 코어별 입력 데이터 크기를 계산했을 때 이 변수의 값보다 작은 경우 파티션의 크기는 이 크기로 맞춰서 만들어짐. 자세한 설명은 stackoverflow 문서 확인
<code>spark.sql.files.minPartitionNum</code>	Default parallelism	이 역시 파일 기반 데이터소스에서만 의미가 있으며 보장되지는 않지만 파일을 읽어들이는 때 최소 몇 개의 파티션을 사용할지 결정하는 변수. 설정되어 있지 않다면 기본값은 <code>spark.default.parallelism</code> 과 동일.
<code>spark.default.parallelism</code>		RDD를 직접 조작하는 경우 의미가 있는 변수로 shuffling 후 파티션의 수를 결정. 단순히 이야기하자면 클러스터 내 총 코어의 수에 해당.

◆ spark.sql.shuffle.partitions

- ❖ 클러스터 자원과 처리 데이터의 크기를 고려해서 잡마다 바꿔서 설정
- ❖ 큰 데이터를 처리하는 거라면 클러스터 전체 코어의 수로 설정
- ❖ AQE를 사용하는 관점에서는 조금더 크게 잡는 것이 좋음

◆ Salting이란?

❖ Salting이란 Skew Partition을 처리하기 위한テクニック

- AQE의 등장으로 인해 그렇게 많이 쓰이지 않음
 - 단 AQE만으로 skew partition 이슈가 사라지지 않는다면 사용이 필요할 수도 있음

❖ 랜덤 필드를 만들고 그 기준으로 파티션을 새로 만들어서 처리

- Aggregation 처리의 경우에는 효과적일 수 있음
- Join의 경우에는 그리 효과적이지 않음
 - AQE에 의존하는 것이 좋음
 - 사실상 Join시 사용 Saltingテクニック을 일반화한 것이 AQE의 skew JOIN 처리방식

❖ [뒤 Salting 데모 github 링크](#)

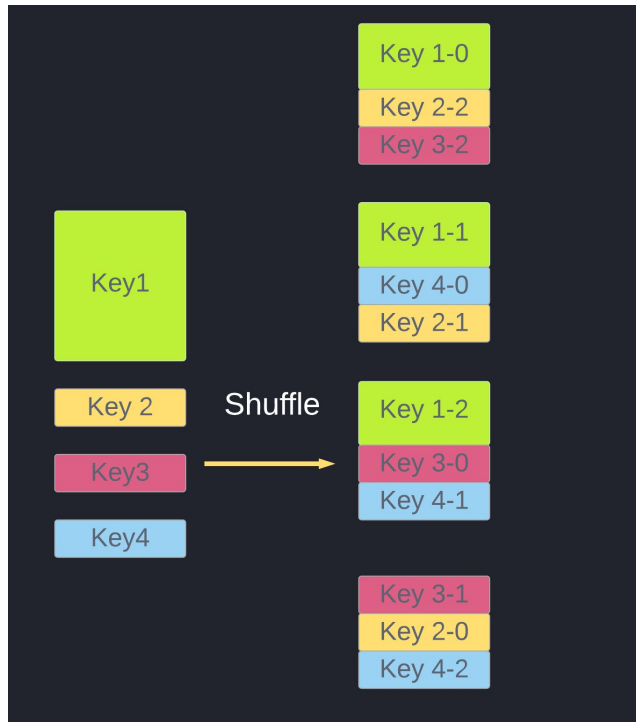
◆ Salting을 Aggregation에 사용한 예 - 원본

```
SELECT item_id, COUNT(1)
FROM sales
GROUP BY 1
ORDER BY 2 DESC
LIMIT 100;
```

item_id	sum(cnt)
100	799999082
9634493	24
6793824	24
17792786	24
25796293	24

◆ Salting을 Aggregation에 사용한 예 (1)

```
SELECT item_id, SUM(cnt)
FROM (
  SELECT item_id, salt, COUNT(1) cnt
  FROM (
    SELECT FLOOR(RAND()*200) salt, item_id
    FROM sales
  )
  GROUP BY 1, 2
)
GROUP BY 1
ORDER BY 2 DESC
LIMIT 100;
```



Source: [Chengzhi Zhao](#)

◆ Salting을 Skew JOIN에 사용한 예 - 원본

```
SELECT date, sum(quantity * price) AS total_sales  
FROM sales s  
JOIN items i ON s.item_id = i.id  
GROUP BY 1  
ORDER BY 2 DESC;
```

◆ Salting을 Skew JOIN에 사용한 예 (1)

```
SELECT date, sum(quantity * price) AS total_sales
```

```
FROM (
```

```
SELECT *, FLOOR(RAND()*20) AS salt  
FROM sales
```

← Skew Partition이 있는 쪽에 salt 추가

```
) s
```

```
JOIN (
```

```
SELECT *, EXPLODE(ARRAY(0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19)) AS salt  
FROM items
```

```
) i ON s.item_id = i.id and s.salt = i.salt
```

```
GROUP BY 1
```

```
ORDER BY 2 DESC;
```

← 반대 Partition은 중복 레코드 만들기

↑
JOIN 조건에 salt
추가하기

◆ Salting을 Skew JOIN에 사용한 예 (2)

```
SELECT date, sum(quantity * price) AS total_sales
FROM (
  SELECT *, CASE WHEN item_id = 100 THEN FLOOR(RAND()*20) ELSE 1 END AS salt
  FROM sales
) s
JOIN (
  SELECT *, EXPLODE(ARRAY(0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19)) AS salt
  FROM items
  WHERE id = 100

  UNION

  SELECT *, 1 AS salt
  FROM items
  WHERE id <> 100
) i ON s.item_id = i.id and s.salt = i.salt
GROUP BY 1
ORDER BY 2 DESC
```

◆ Spill이란 무엇인가?

- ❖ 파티션의 크기가 너무 커서 메모리가 부족한 경우 그 여분을 디스크에 쓰는 것
 - Spill이 발생하면 실행시간이 늘어나고 OOM이 발생할 가능성이 올라감
- ❖ Spill이 발생하는 경우 (즉 메모리가 부족해지는 경우)
 - Skew Partition 대상 Aggregation
 - Skew Partition 대상 JOIN
 - 굉장히 큰 explode 작업
 - 큰 파티션(`spark.sql.files.maxPartitionBytes`)이라면 위의 작업시 spill 가능성이 더 높아짐



◆ Spill의 종류

❖ Spill (memory)

- 디스크로 spill된 데이터가 메모리에 있을 때의 크기
- Deserialized 형태라 크기가 보통 8-10배 정도 더 큼

❖ Spill (disk)

- 메모리에서 spill된 데이터가 디스크에서 차지하는 크기
- Serialized 형태라 보통 크기가 훨씬 더 작음

▼ Aggregated Metrics by Executor

Show entries

Executor ID	Logs	Address	Task Time
driver		192.168.0.16:59235	11 min

...

결국 spill (memory)와 spill (disk)는 같은 데이터를 가르키는데 하나는 메모리에 있을 때의 크기이고 다른 하나는 디스크에 있을 때의 크기

보통 오래 걸리는 태스크들은 spill이 있음

Search:

Input Size / Records	Shuffle Write Size / Records	Spill (Memory)	Spill (Disk)
1.3 GiB / 10000000000	1.8 GiB / 199887914	17 GiB	2.6 GiB

All about Partitions

셔플링 후 생기는 파티션만 중요한 것이 아니라 데이터 로딩과 저장시 파티션의 수도 중요하다. 이에 대해 알아보자

◆ 세 종류의 Partition (Life cycle of Partitions)

- a. 입력 데이터를 로드할 때 파티션 수와 크기
- b. 셔플링 후 만들어지는 파티션 수와 크기
- c. 데이터를 최종적으로 저장할 때 파티션 수와 크기

◆ 파티션의 크기는 128MB - 1GB가 좋음

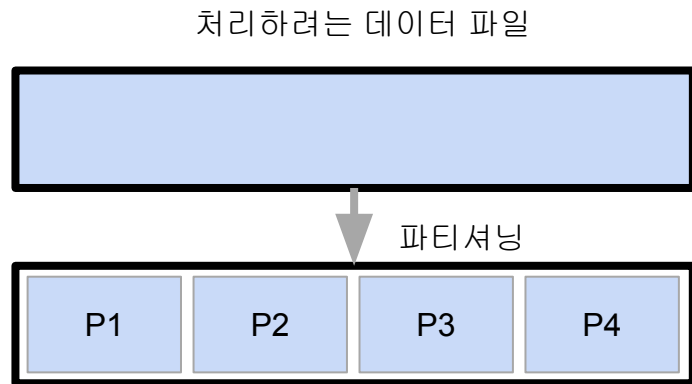
◆ 입력 데이터를 로드할 때 파티션 수와 크기

❖ 기본적으로는 파티션의 최대 크기에 의해 결정

- `spark.sql.files.maxpartitionbytes` (128MB)

❖ 결국 해당 데이터가 어떻게 저장되었는지와 연관이 많이 됨

- 파일포맷이 무엇이고 압축되었는지? 압축되었다면 무슨 알고리즘으로?
 - 결국 **Splittable**한가 -> 한 큰 파일을 다수의 파티션으로 나눠 로드할 수 있는가?
- 기타 관련 Spark 환경변수들



◆ 입력 데이터 파티션 수와 크기를 결정해주는 변수들 (1)

❖ bucketBy로 저장된 데이터를 읽는 경우

- Bucket의 수와 Bucket 기준 컬럼들과 정렬 기준 컬럼들

❖ 읽어들이는 데이터 파일이 **splittable**한지?

- PARQUET/AVRO등이 좋은 이유가 됨: 항상 **splittable**
- JSON/CSV등의 경우 한 레코드가 **multi-line**이라면 **splittable**하지 않음
 - Single line이라도 압축시 **bzip2**를 사용해야만 **splittable**

❖ 입력 데이터의 전체 크기 (모든 파일크기의 합)

- 입력 데이터를 구성하는 파일의 수

❖ 내가 리소스 매니저에게 요청한 **CPU**의 수

- **executor**의 수 x **executor** 별 **CPU** 수

◆ 입력 데이터 파티션 수와 크기를 결정해주는 변수들 (2)

❖ 관련 환경 변수들 (앞에서 한번 살펴보았음)

환경 변수 이름	기본값	설명
<code>spark.sql.files.maxPartitionBytes</code>	128MB	파일 기반 데이터 소스(Parquet, JSON, ORC 등등)에서 데이터프레임을 만들 때 한 파티션의 최대 크기
<code>spark.sql.files.openCostInBytes</code>	4MB	이 역시 파일 기반 데이터소스에서만 의미가 있으며 입력 데이터를 읽기 위해 몇 개의 파티션을 사용할지 결정하는데 사용됨. 대부분의 경우 별 의미 없는 변수이지만 기본적으로 코어별 입력 데이터 크기를 계산했을 때 이 변수의 값보다 작은 경우 파티션의 크기는 이 크기로 맞춰서 만들어짐. 자세한 설명은 stackoverflow 문서 확인
<code>spark.sql.files.minPartitionNum</code>	Default parallelism	이 역시 파일 기반 데이터소스에서만 의미가 있으며 보장되지는 않지만 파일을 읽어들이는 때 최소 몇 개의 파티션을 사용할지 결정하는 변수. 설정되어 있지 않다면 기본값은 <code>spark.default.parallelism</code> 과 동일.
<code>spark.default.parallelism</code>		RDD를 직접 조작하는 경우 의미가 있는 변수로 shuffling 후 파티션의 수를 결정. 단순히 이야기하자면 클러스터 내 총 코어의 수에 해당.

◆ 입력 데이터 파티션 수와 크기 결정 방식

❖ 먼저 아래 공식으로 maxSplitBytes를 결정

- $\text{bytesPerCore} = (\text{데이터파일들 전체크기} + \text{파일수} * \text{OpenCostInBytes}) / \text{default.parallelism}$
- $\text{maxSplitBytes} = \text{Min}(\text{maxPartitionBytes}, \text{bytesPerCore})$
 - $\text{maxSplitBytes} = \text{Min}(\text{maxPartitionBytes}, \text{Max}(\text{bytesPerCore}, \text{OpenCostInBytes}))$

❖ 입력 데이터를 구성하는 각 파일에 대해 다음을 진행

- Splittable하다면 maxSplitBytes 단위로 분할하여 File Chunk 생성
- Splittable하지 않거나 크기가 maxSplitBytes보다 작다면 하나의 File chunk 생성

❖ 다음으로 위에서 만들어진 File Chunk들로부터 파티션 생성

- 기본적으로 한 파티션은 하나 혹은 그 이상의 File Chunk들로 구성
- 한 파티션에 다음 File Chunk의 크기 + openCostInBytes를 더했을 때 이 값이 maxSplitBytes를 넘어가지 않을 때까지 계속해서 머지함 (파일들이 하나의 파티션으로 패킹됨)

◆ Spark Scala 코드 보기: maxSplitBytes

```
86 def maxSplitBytes(  
87     sparkSession: SparkSession,  
88     selectedPartitions: Seq[PartitionDirectory]): Long = {  
89     val defaultMaxSplitBytes = sparkSession.sessionState.conf.filesMaxPartitionBytes  
90     val openCostInBytes = sparkSession.sessionState.conf.filesOpenCostInBytes  
91     val minPartitionNum = sparkSession.sessionState.conf.filesMinPartitionNum  
92         .getOrElse(sparkSession.leafNodeDefaultParallelism)  
93     val totalBytes = selectedPartitions.flatMap(_.files.map(_.getLen + openCostInBytes)).sum  
94     val bytesPerCore = totalBytes / minPartitionNum  
95  
96     Math.min(defaultMaxSplitBytes, Math.max(openCostInBytes, bytesPerCore))  
97 }
```

◆ Spark Scala 코드 보기: 파티션 생성 코드

```
72     val openCostInBytes = sparkSession.sessionState.conf.filesOpenCostInBytes
73     // Assign files to partitions using "Next Fit Decreasing"
74     partitionedFiles.foreach { file =>
75         if (currentSize + file.length > maxSplitBytes) {
76             closePartition()
77         }
78         // Add the given file to the current partition.
79         currentSize += file.length + openCostInBytes
80         currentFiles += file
81     }
82     closePartition()
```

◆ 입력 데이터 파티션 수와 크기 결정 방식 예 1

- ❖ 파일 수: 50
- ❖ 파일 포맷: PARQUET
- ❖ 파일 크기: 65MB
- ❖ Spark Application에 할당된 코어수: 10
- ❖ `spark.sql.files.maxPartitionBytes`: 128MB
- ❖ `spark.default.parallelism`: 10
- ❖ `spark.sql.files.openCostInBytes`: 4MB

- $\text{bytesPerCore} = (50 \times 65\text{MB} + 50 \times 4\text{MB}) / 10 = 345\text{MB}$
- $\text{maxSplitBytes} = \text{Min}(128\text{MB}, 345\text{MB}) = 128\text{MB}$
- 최종 파티션수: 50

◆ 입력 데이터 파티션 수와 크기 결정 방식 예 2

❖ 파일 수: 50

❖ 파일 포맷: PARQUET

❖ 파일 크기: 63MB

❖ Spark Application에 할당된 코어수: 10

❖ `spark.sql.files.maxPartitionBytes`: 128MB

❖ `spark.default.parallelism`: 10

❖ `spark.sql.files.openCostInBytes`: 4MB

- $\text{bytesPerCore} = (50 \times 63\text{MB} + 50 \times 4\text{MB}) / 10 = 335\text{MB}$

- $\text{maxSplitBytes} = \text{Min}(128\text{MB}, 335\text{MB}) = 128\text{MB}$

- 최종 파티션수: 50

- 25개일꺼라 생각할 수 있지만 `openCostInBytes` 때문에 패킹불가

◆ 입력 데이터 파티션 수와 크기 결정 방식 예 3

- ❖ 파일 수: 50
- ❖ 파일 포맷: PARQUET
- ❖ 파일 크기: 40MB
- ❖ Spark Application에 할당된 코어수: 10
- ❖ `spark.sql.files.maxPartitionBytes`: 128MB
- ❖ `spark.default.parallelism`: 10
- ❖ `spark.sql.files.openCostInBytes`: 4MB

- $\text{bytesPerCore} = (50 \times 40\text{MB} + 50 \times 4\text{MB}) / 10 = 220\text{MB}$
- $\text{maxSplitBytes} = \text{Min}(128\text{MB}, 220\text{MB}) = 128\text{MB}$
- 최종 파티션수: 17
 - 한 파티션에 3개 파일까지 패킹 가능

```
partitionedFiles.foreach { file =>
  if (currentSize + file.length > maxSplitBytes) {
    closePartition()
  }
  // Add the given file to the current partition.
  currentSize += file.length + openCostInBytes
  currentFiles += file
}
```

◆ 입력 데이터 파티션 수와 크기 결정 방식 예 4

❖ 파일 수: 10

❖ 파일 포맷: CSV (splittable하지 않은 gzip으로 압축)

❖ 파일 크기: 335MB

❖ Spark Application에 할당된 코어수: 10

❖ spark.sql.files.maxPartitionBytes: 128MB

❖ spark.default.parallelism: 10

❖ spark.sql.files.openCostInBytes: 4MB

- $\text{bytesPerCore} = (10 \times 335\text{MB} + 10 \times 4\text{MB}) / 10 = 339\text{MB}$

- $\text{maxSplitBytes} = \text{Min}(128\text{MB}, 339\text{MB}) = 128\text{MB}$

- 최종 파티션 수: 10

- 파일포맷이 **splittable**하지 않고 크기가 이미 **maxSplitBytes**보다 크기 때문에 패킹이 없음
- 파티션 하나의 크기가 너무 크게 됨!

◆ 입력 데이터 파티션 수와 크기 결정 방식 예 5

❖ 파일 수: 10

❖ 파일 포맷: CSV (splittable한 bzip2으로 압축)

❖ 파일 크기: 311MB

❖ Spark Application에 할당된 코어수: 10

❖ spark.sql.files.maxPartitionBytes: 128MB

❖ spark.default.parallelism: 10

❖ spark.sql.files.openCostInBytes: 4MB

- $\text{bytesPerCore} = (10 \times 311\text{MB} + 10 \times 4\text{MB}) / 10 = 315\text{MB}$

- $\text{maxSplitBytes} = \text{Min}(128\text{MB}, 315\text{MB}) = 128\text{MB}$

- 최종 파티션수: 25

- 파일포맷이 **splittable**하기 때문에 각 파일은 3개의 File Chunk로 나뉨 (128, 128, 55)

- 중요한 포인트는 각 파티션은 로드되면서 압축이 풀리며 그래서 **PARQUET** 파일이 좋은 포맷임

◆ 입력 데이터 파티션 수와 크기 결정 방식 예 6

❖ 파일 수: 100

❖ 파일 포맷: CSV (splittable하지 않은 gzip으로 압축)

❖ 파일 크기: 10MB

❖ Spark Application에 할당된 코어수: 10

❖ spark.sql.files.maxPartitionBytes: 128MB

❖ spark.default.parallelism: 10

❖ spark.sql.files.openCostInBytes: 4MB

- $\text{bytesPerCore} = (100 \times 10\text{MB} + 100 \times 4\text{MB}) / 10 = 140\text{MB}$
- $\text{maxSplitBytes} = \text{Min}(128\text{MB}, 140\text{MB}) = 128\text{MB}$
- 최종 파티션수: 12
 - splittable하지는 않지만 크기가 작아서 파일 9개까지 128MB에 채울 수 있음 $(10+4) \times 9 = 126$
 - 마지막 파일 하나는 그대로 파티션 하나가 됨

◆ 입력 데이터 파티션 수와 크기 결정 방식 요약

❖ 앞서 공식을 다시 보자

- $\text{bytesPerCore} = (\text{데이터파일들 전체 크기} + \text{파일 수} * \text{openCostInBytes}) / \text{default.parallelism}$
- $\text{maxSplitBytes} = \text{Min}(\text{maxPartitionBytes}, \text{max}(\text{bytesPerCore}, \text{openCostInBytes}))$

❖ 요약하자면

- 코어별로 처리해야 할 크기가 너무 크다면 **maxPartitionBytes**로 제약
- 데이터의 크기가 적당하면 코어수 만큼의 파티션을 생성
- 데이터의 크기가 너무 작으면 **openCostInBytes**로 설정한 크기로 파티션 생성

❖ 간단 데모

◆ Bucketing ([bucketBy](#)) 소개

- ❖ 데이터를 자주 사용되는 컬럼 기준으로 미리 저장해두고 활용
- ❖ 다양한 최적화 가능
 - 조인 대상 테이블들이 조인 키를 갖고 bucketing된 경우 shuffle free join 가능
 - 한쪽만 bucketing되어있는 경우 one-side shuffle free join 가능 (bucket의 크기에 달림)
 - Bucket pruning를 통한 최적화 가능
 - Shuffle free aggregation
- ❖ Bucket 정보가 metastore에 저장되고 Spark Compiler는 이를 활용
 - 이 경우 sortBy를 통해 순서를 미리 정해주기도 함
 - Spark 테이블로 저장하고 로딩해야지만 이 정보를 이용 가능함
 - saveAsTable, spark.table()

◆ Bucketing ([bucketBy](#)) 저장 방식

❖ Bucket의 수 x Partition의 수 만큼의 파일이 만들어짐

- 예) DataFrame의 Partition 수가 10이고 Bucket의 수가 4라면 40개의 파일 생성
- 다시 읽어들일 때 10개의 Partition으로 읽혀짐

❖ 다시 읽어들일 때 원래 Partition의 수만큼으로 재구성됨

❖ Bucketing 키를 기반으로 작업시 셔플이 없어짐

❖ Bucketing 데모

- [Bucket 테이블 저장하기](#)
- [Bucket 테이블 읽기](#)

◆ Small Files 신드롬이란?

❖ 왜 작은 크기의 많은 파일이 문제가 되는가?

- 64MB의 파일 하나를 읽는 것 vs. 64Byte의 파일 백만개를 읽는 것
- 이 API 콜은 모두 네트워크 RPC 콜
- 파일 시스템 접근 관련 오버헤드
 - 파일 하나를 접근하기 위해서 다수의 API 콜이 필요
 - 그래서 앞서 `openCostInBytes`라는 오버헤드가 각 파일마다 부여됨

❖ 읽어들이면서 파티션의 수를 줄일 수 있지만 오버헤드가 큼

- 파일로 쓸 때 어느 정도 정리를 해주는 것이 필요

◆ JSON으로 저장된 데이터 3가지 읽어보기

❖ 파일 수: 100

❖ 파일 포맷: JSON

- 압축여부: No, Yes (gzip), Yes (bzip2)

❖ 각 파일 크기

- 10.5MB, 955KB, 481KB

❖ Spark Application에 할당된 코어수: 10

❖ `spark.sql.files.maxPartitionBytes`: 128MB

❖ `spark.default.parallelism`: 10

❖ `spark.sql.files.openCostInBytes`: 4MB

❖ 데모 보기

- [테스트 데이터 저장하기](#)
- [로드된 데이터 파티션 수 알아보기](#)

bytesPerCore = 145MB
maxSplitBytes = 128MB

$128/14.5 = 9$ file chunks / partition
12개 파티션

bytesPerCore = 49.5MB = $(95.5+400)/10$
maxSplitBytes = 49.5MB

$49.5/4.95 = 10$ file chunks / partition
10개 파티션

bytesPerCore = 44.8MB = $(48.1+400)/10$
maxSplitBytes = 44.8MB

$44.8/4.48 = 10$ file chunks / partition
10개 파티션

◆ 셔플링 후 만들어지는 파티션 수와 크기

- ❖ `spark.sql.shuffle.partitions`
- ❖ AQE를 꼭 사용
- ❖ 이에 대해서는 충분히 이야기했음

◆ 데이터를 저장할 때 파티션 수와 크기

❖ 3가지 방식이 존재

- 기본
- bucketBy
- partitionBy

❖ 파일크기를 레코드 수로도 제어 가능

- `spark.sql.files.maxRecordsPerFile`
 - 기본값은 0이며 레코드수로 제약하지 않는다는 의미

◆ 데이터를 저장할 때 파티션 수와 크기: 기본

❖ bucketBy나 partitionBy를 사용하지 않는 경우

- 각 파티션이 하나의 파일로 쓰여짐
- saveAsTable vs. save

❖ 적당한 크기와 수의 파티션을 찾는 것이 중요

- 작은 크기의 다수의 파티션이 있다면 문제
- 큰 크기의 소수의 파티션도 문제 (splittable하지 않은 포맷으로 저장이 될 경우)

❖ Repartition 혹은 coalesce를 적절히 사용

- 이 경우 AQE의 Coalescing가 도움이 될 수 있음 (repartition)

❖ PARQUET 포맷 사용

- Snappy compression 사용

◆ 데이터를 저장할 때 파티션 수와 크기: **bucketBy**

❖ 앞에서 이미 살펴봄

❖ 데이터 특성을 잘 아는 경우 특정 **ID**를 기준으로 나눠서 테이블로 저장

- 다음부터는 이를 로딩하여 사용함으로써 반복 처리시 시간 단축
 - **Bucket**의 수와 기준 **ID** 지정
- 데이터의 특성을 잘 알고 있는 경우 사용 가능

❖ **bucket**의 수와 키를 지정해야함

- `df.write.mode("overwrite").bucketBy(3, key).saveAsTable(table)`
- **sortBy**를 써서 순서를 정하기도 함
- 이 정보는 **metastore**에 같이 저장됨

```
CREATE TABLE bucketed_table (  
  id INT,  
  name STRING,  
  age INT  
)  
USING PARQUET  
CLUSTERED BY (id)  
INTO 4 BUCKETS;
```

◆ 데이터를 저장할 때 파티션 수와 크기: **partitionBy**

❖ 굉장히 큰 로그 파일을 데이터 생성시간 기반으로 데이터 읽기를 많이 한다면?

- 데이터 자체를 연도-월-일의 폴더 구조로 저장
 - 이를 통해 데이터를 읽기 과정을 최적화 (스캐닝 과정이 줄어들거나 없어짐)
 - 데이터 관리도 쉬워짐 (**Retention Policy** 적용시)
- 하지만 **Partition key**를 잘못 선택하면 엄청나게 많은 파일들이 생성됨!
 - **Cardinality**가 높은 컬럼을 키로 사용하면 안됨

❖ **partitioning**할 키를 지정해야함

- `df.write.mode("overwrite").partitionBy("order_month").saveAsTable("order")`
- `df.write.mode("overwrite"). \`
`partitionBy("year", "month", "day"). \`
`saveAsTable("appl_stock")`

```
CREATE TABLE partitioned_table (  
  id INT,  
  value STRING  
) USING parquet  
PARTITIONED BY (id);
```

◆ 데이터를 저장할 때 파티션 수와 크기: bucketBy & partitionBy

❖ partitionBy 후에 bucketBy 사용

❖ 필터링 패턴 기준 partitionBy 후 그룹핑/조인 패턴 기준 bucketBy 사용

```
df.write.mode("overwrite").partitionBy("dept").bucketBy(5, "employeeId")
```


◆ Spark Persistent 테이블의 통계 정보 확인

❖ spark.sql("DESCRIBE EXTENDED 테이블이름")

- 이를 통해 bucket/partition 테이블 정보를 얻을 수 있음

spark.sql("DESCRIBE EXTENDED appl_stock").show()

col_name	data_type	comment
id	bigint	null
# Detailed Table ...		
Database	default	
Table	spark_bucket_table1	
Created Time	Mon Feb 27 19:25:...	
Last Access	UNKNOWN	
Created By	Spark 3.3.1	
Type	MANAGED	
Provider	parquet	
Num Buckets	100	
Bucket Columns	[`id`]	
Sort Columns	[]	
Statistics	44115/5 bytes	

요약

- Dynamically optimizing skew joins
- Salting을 통한 Data Skew 처리
- All about Partitions



Grepp Inc

한기용

keeyonghan@hotmail.com